

SOFTWARE ENGINEERING · CASE STUDY NO. 38

CarbonLedger

Design Pack

Temporal data model, class diagram and reporting cycle activity diagram

CL-DSG-001 · VERSION 1.0 · DRAFT FOR REVIEW

2

time axes

5

diagrams

SHA-256

report hash

Prepared by: Rohan Vashisht

Cohort: Jensen Huang

Roll No.: 150096724132

Programme: B.Tech CSE 2024–28 · Semester V

Project: Emissions Accounting for a Multi-Plant Manufacturer

Project website: <https://carbon-ledger-rohan.vercel.app/>

Figures stated as project data come from the case study. Numbers in worked examples and test fixtures are synthetic and are not published emission factors.

Summary & Contents

Purpose of this document

Describes the architecture and the temporal reference data design that makes historical reproduction possible by construction, with the class diagram, data model and reporting cycle workflow.

Contents

1	Design overview	4
2	Temporal reference data design	5
3	Class diagram	9
4	Activity diagram: reporting cycle	10
5	Behaviour design	13
6	Design decisions and trade-offs	14
7	Traceability	14

Document control

Revision history

Version	Date	Description	Author (role)
1.0	03/10/2026	Initial release for review	Sole Engineer

Approvals

Role	Signature	Date
Sponsor (Manufacturer, Head of Sustainability)		
Reporting Lead (Manufacturer)		
Sole Engineer (all project roles)		
Reviewer (Faculty)		

Related documents

CL-SRS-001 Software Requirements Specification (v1.0)

CL-DTT-001 Decision Table and Test Set (v1.0)

CL-DSG-001 Design Pack (v1.0) · this document

CL-CMP-001 Configuration Management Plan (v1.0)

CL-DQP-001 Data Quality Plan (v1.0)

CL-PPL-001 Project Plan (v1.0)

Notice. This document is controlled; printed copies are uncontrolled. Figures stated as project data (data quality tiers of $\pm 2\%$, $\pm 8\%$ and $\pm 25\%$, nine plants, three scopes) come from the case study. Numbers in worked examples and test fixtures are synthetic, are labelled as such, and are not emission factors published by any authority.

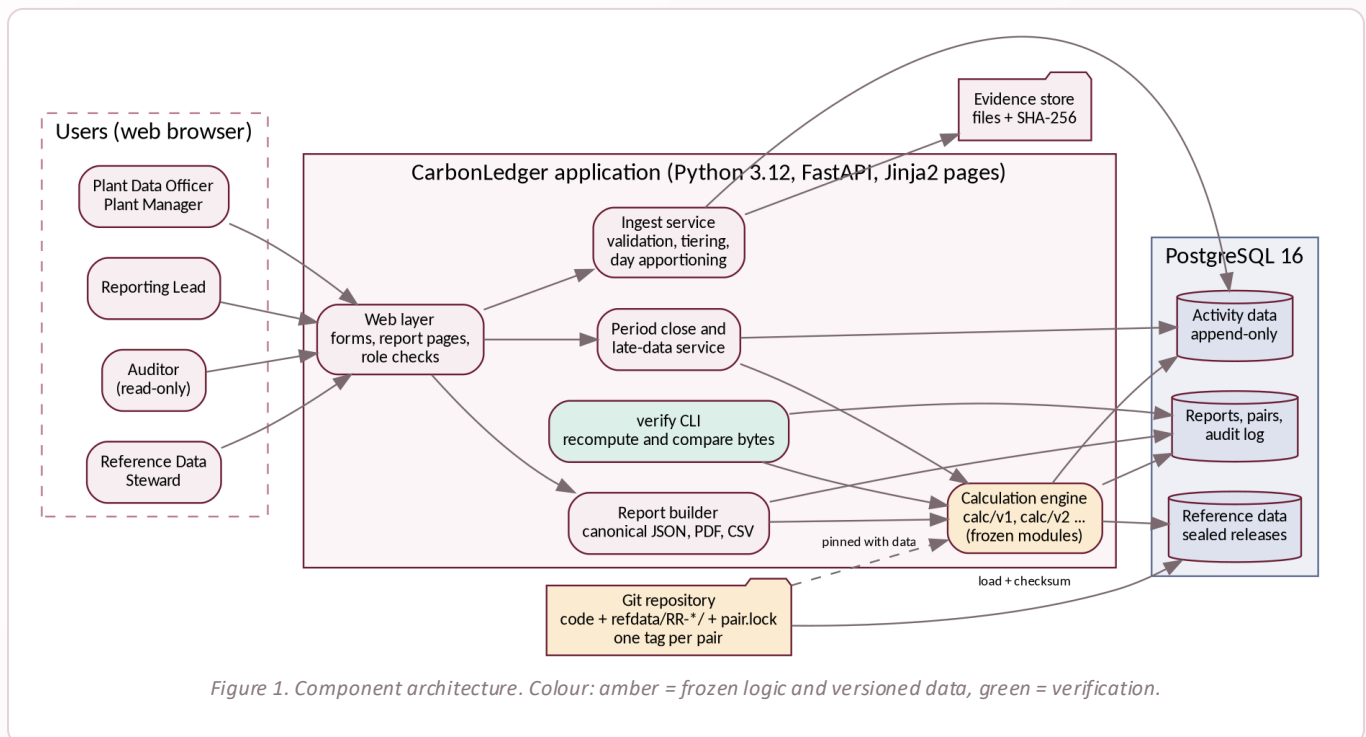
1. Design overview

The design has one governing idea: **history cannot be changed, so reproduction is possible by construction**. Four things are therefore immutable once created: activity records (append-only), reference releases (sealed), frozen calculation modules, and issued reports. Everything else is derived from them by a pure function:

● $\text{report} = f(\text{records as of cut-off, reference release, calc module})$

If the three arguments are identical, the output bytes are identical. The design pack shows how the data model, classes and workflow enforce that.

1.1 Architecture



Component	Responsibility	Requirements
Web layer	Forms, report pages, role checks. Server-rendered, no client framework.	FR-AC-01, NFR-USE-01
Ingest service	Validates, assigns tier from evidence, apportions bills across months, stores append-only.	FR-IN-01 to 07
Calculation engine	Pure functions. Selects the factor, converts units, computes emission and uncertainty in decimals. One frozen module per calc release.	FR-CALC, FR-UNC
Period close and late-data service	State machine for periods, gap-fill, late record handling, restatement.	FR-PC, FR-LATE
Report builder	Produces canonical JSON and hash, then PDF and CSV.	FR-RPT
verify CLI	Recomputes any issued report from its Pair and compares bytes.	FR-RPT-04
PostgreSQL	Stores records, sealed releases, reports, audit log; enforces immutability.	FR-IN-02, FR-FAC-01, 04

1. Design overview

Component	Responsibility	Requirements
Git repository	Holds code and <code>refdata/RR-*/</code> together; one tag per Pair.	CL-CMP-001
Evidence store	Files stored by SHA-256; the hash is kept on the record.	FR-IN-01

1.2 Module and repository layout

```
carbonledger/  
  src/carbonledger/  
    web/          routes, templates, role checks  
    ingest/       validation, tiering, apportion.py  
    calc/           
      v1/         FROZEN once a report is issued with it  
        engine.py factor lookup, conversion, emission, uncertainty  
        rounding.py ROUND_HALF_EVEN, quantise to 6 dp  
      v2/         created only when a change would alter any number  
    close/        period state machine, gapfill.py, late.py, restate.py  
    report/       canonical.py (JSON bytes), render_pdf.py, render_csv.py  
    verify/       cli.py  
  refdata/  
    RR-2024.1/    factors.csv, conversions.csv, policy.json, manifest.json  
    RR-2025.1/    ...  
  migrations/    SQL: tables, triggers, constraints  
  tests/         unit, integration, decision_table, golden/R-<id>/  
  pairs/         CL-2024.1.lock, CL-2025.1.lock ...
```

2 Temporal reference data design

2.1 Two time axes

Every fact that matters has two kinds of time. Keeping them separate is what makes the system reproducible and still able to accept corrections.

Axis	Meaning	Emission factor	Activity record
Valid time	When the fact applies in the real world	<code>valid_from</code> , <code>valid_to</code> (the activity dates the factor covers)	<code>activity_start</code> , <code>activity_end</code>
Record time	When the system learned the fact	<code>loaded_at</code> , and the release it belongs to	<code>recorded_at</code>

Corrections never overwrite: they add a new row with later record time. A report is a question asked at one record-time instant (`closed_at`) about valid-time data, answered using one release. Because rows with earlier record time never change, the same question always gets the same answer.

2. Temporal reference data design

2.2 Factor selection rule

For a record with activity date d the factor is the one row of the pinned release with matching type and unit and $\text{valid_from} \leq d < \text{valid_to}$. The interval is half-open so that 2024-12-31 and 2025-01-01 can never both match. Overlaps are impossible inside a release (exclusion constraint), and no match is an error. There is **no "latest" fallback**.

2.3 Worked example (synthetic test factors)

Time	Event	What exists
Jan 2025	FY2024 report issued, pinned to RR-T2024.1 (diesel 2024 = 2.500000).	R-T2024-A, hash H1.
Feb 2025	Authority publishes 2025 factors. Release RR-T2025.1 sealed (2024 and 2025 rows).	New reports use it for 2025 activity.
Mar 2026	Authority corrects the 2024 diesel factor to 2.550000. Release RR-T2026.1 sealed.	RR-T2024.1 and RR-T2025.1 untouched.
Oct 2026	verify R-T2024-A	Uses RR-T2024.1 and the stored inputs as of <code>closed_at</code> . Output hash is H1 again.
Oct 2026	Reporting Lead wants the corrected figure	Runs what-if with RR-T2026.1 (stamped COMPARISON), then decides on restatement; a new version R-T2024-A v2 is created, v1 kept.

2. Temporal reference data design

2.4 Entity relationship model

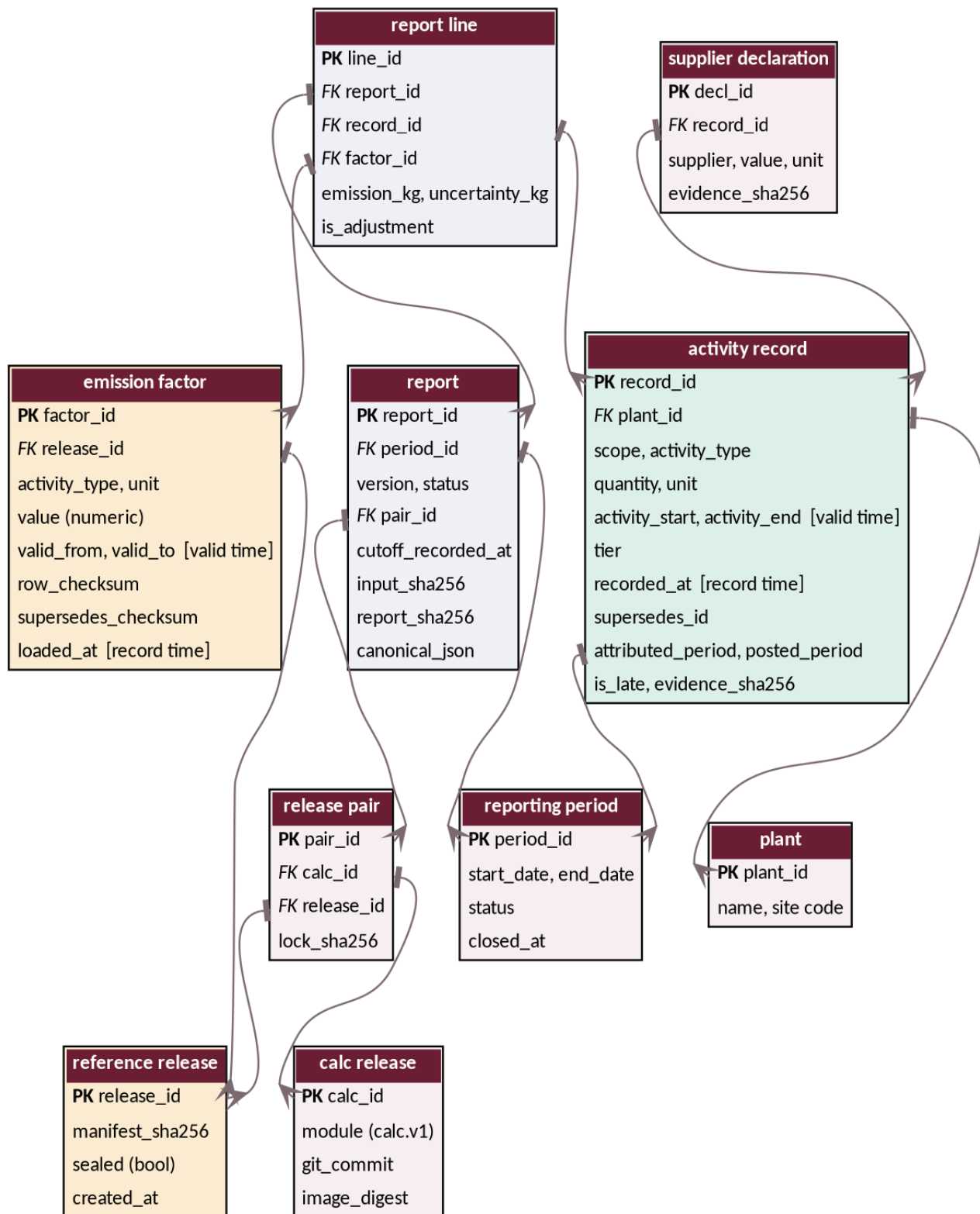


Figure 2. Data model. Green = input data, yellow = reference data, slate = report data.

2. Temporal reference data design

2.5 Key database rules

```
-- Valid-time interval, no overlap inside one release (exclusion constraint)
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE emission_factor (
  factor_id      bigserial PRIMARY KEY,
  release_id     text NOT NULL REFERENCES reference_release(release_id),
  activity_type  text NOT NULL,
  unit          text NOT NULL,
  value         numeric(20,10) NOT NULL,
  valid         daterange NOT NULL,           -- [valid_from, valid_to)
  row_checksum  char(64) NOT NULL,
  supersedes_checksum char(64),
  loaded_at     timestampz NOT NULL DEFAULT now(),
  EXCLUDE USING gist (release_id WITH =, activity_type WITH =, unit WITH =, valid WITH &&)
);

-- Sealed releases and all activity records are append-only
CREATE FUNCTION block_change() RETURNS trigger AS $$
BEGIN RAISE EXCEPTION 'immutable: % on %', TG_OP, TG_TABLE_NAME; END $$ LANGUAGE plpgsql;
CREATE TRIGGER activity_immutable BEFORE UPDATE OR DELETE ON activity_record
  FOR EACH ROW EXECUTE FUNCTION block_change();
-- emission_factor: trigger rejects INSERT/UPDATE/DELETE when its release is sealed.
```

The application database role has no right to disable these triggers; migrations run under a separate role (CL-CMP-001).

2.6 Why not the alternatives

Alternative	Problem
One factor table, overwrite when revised	Destroys history; 2024 reports change in 2026.
History rows with a "current" flag	Lookup depends on a flag that can be flipped; reproduction depends on when the query runs.
Membership table linking releases to shared rows	Works, but the no-overlap rule cannot be a native constraint; more code to get wrong. Copy-on-release costs only a few hundred rows per year.
Snapshot all data per report	Heavy; the cut-off rule gives the same result with one timestamp and a hash check.

3. Class diagram

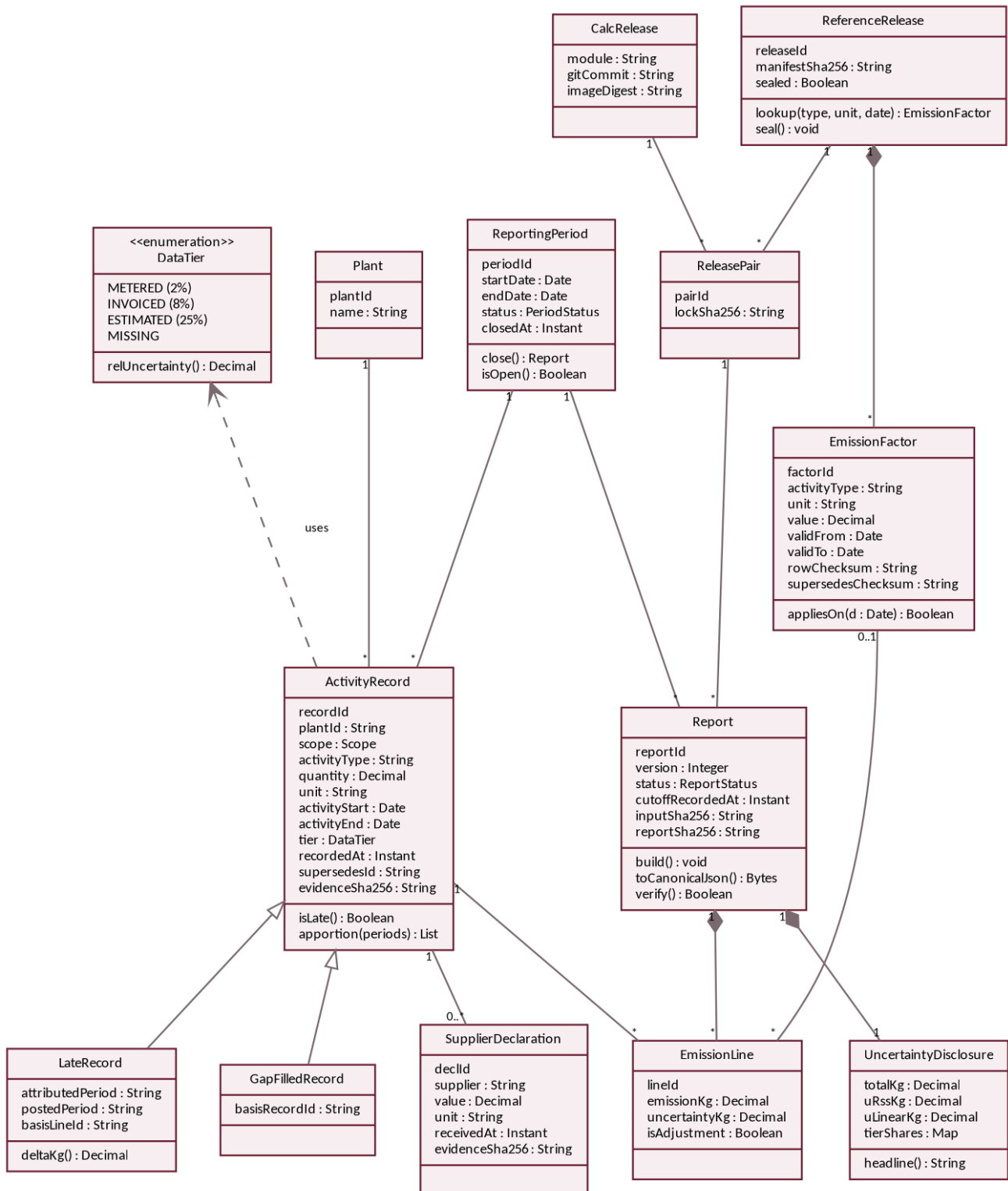


Figure 3. Class diagram for activity data, emission factor and report, with the supporting classes. Open triangle = generalisation, filled diamond = composition.

3. Class diagram

Class	Role and notable rule
ActivityRecord	Immutable after creation. <code>isLate()</code> is true when the period of its activity date was CLOSED at <code>recordedAt</code> . <code>apportion()</code> splits month-crossing intervals by days.
LateRecord, GapFilledRecord	Specialisations carrying the extra data each case needs (attributed and posted period; basis record).
SupplierDeclaration	Evidence only; may supply a value for a missing Scope 3 item as tier ESTIMATED.
EmissionFactor	One row of a release; <code>appliesOn(d)</code> tests the half-open interval.
ReferenceRelease	Sealed set of factors, conversions, tier percentages and rounding policy. <code>lookup()</code> returns exactly one factor or raises.
CalcRelease, ReleasePair	The frozen module and the binding of module and release into one baseline.
DataTier	Enumeration with the project percentages: 2%, 8%, 25%.
ReportingPeriod	Holds status and <code>closedAt</code> ; <code>close()</code> is the only way to produce a report R1.
Report	Holds the Pair, cut-off, input hash and report hash; <code>toCanonicalJson()</code> defines the bytes; <code>verify()</code> recomputes.
EmissionLine	One line per record: quantity, factor value, emission, uncertainty, adjustment flag.
UncertaintyDisclosure	Totals, RSS, worst-case bound and tier shares; <code>headline()</code> formats "E ± U (x%)".

4

Activity diagram: reporting cycle

The cycle has two parts. Colours show the actor: blue = Plant Data Officer or Plant Manager, yellow = system, green = Reporting Lead, red = exception path, rose = Auditor. Part A ends where part B begins.

4. Activity diagram: reporting cycle

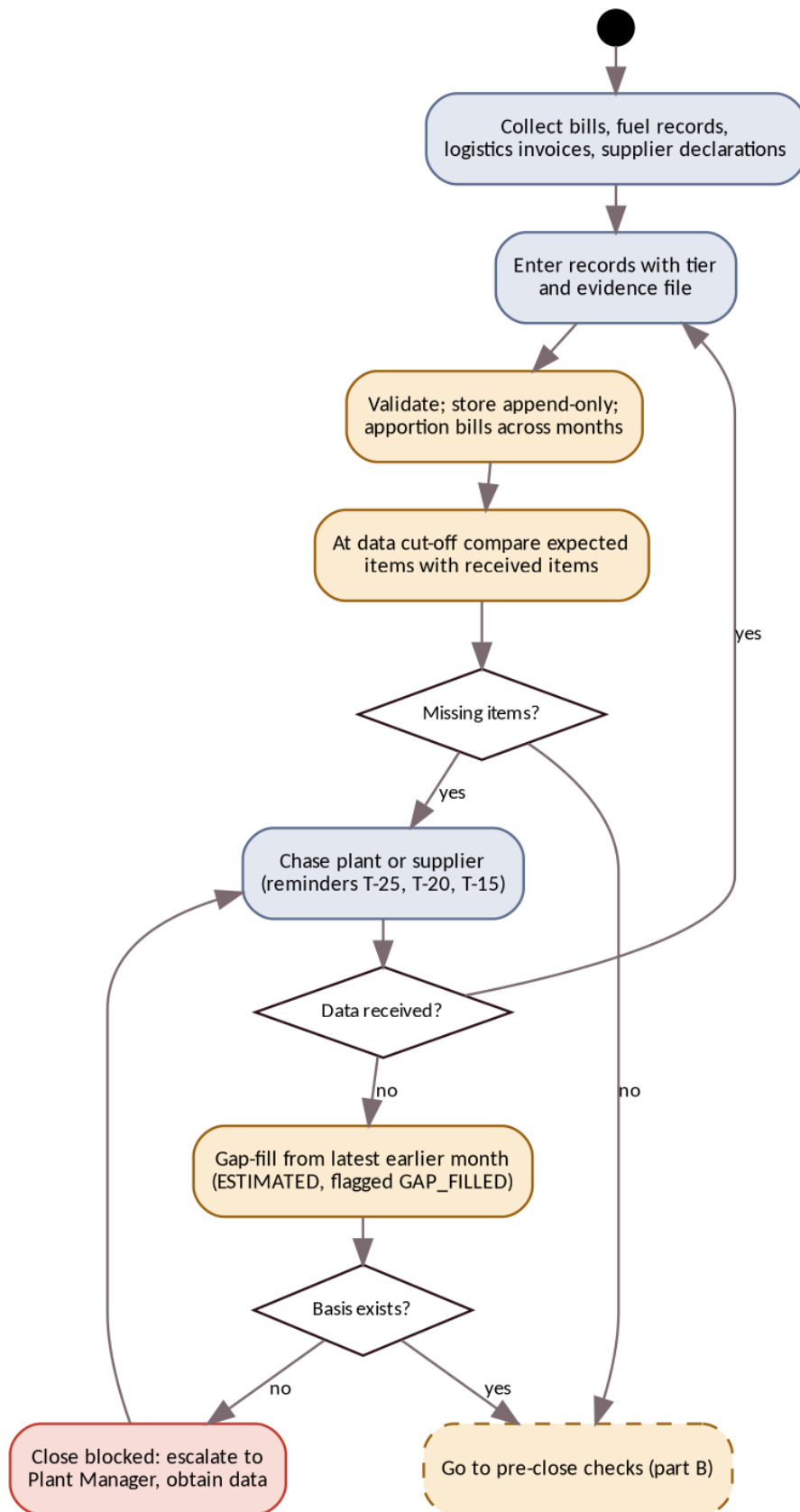


Figure 4a. Reporting cycle, part A: collection, completeness, gap-fill.

4. Activity diagram: reporting cycle

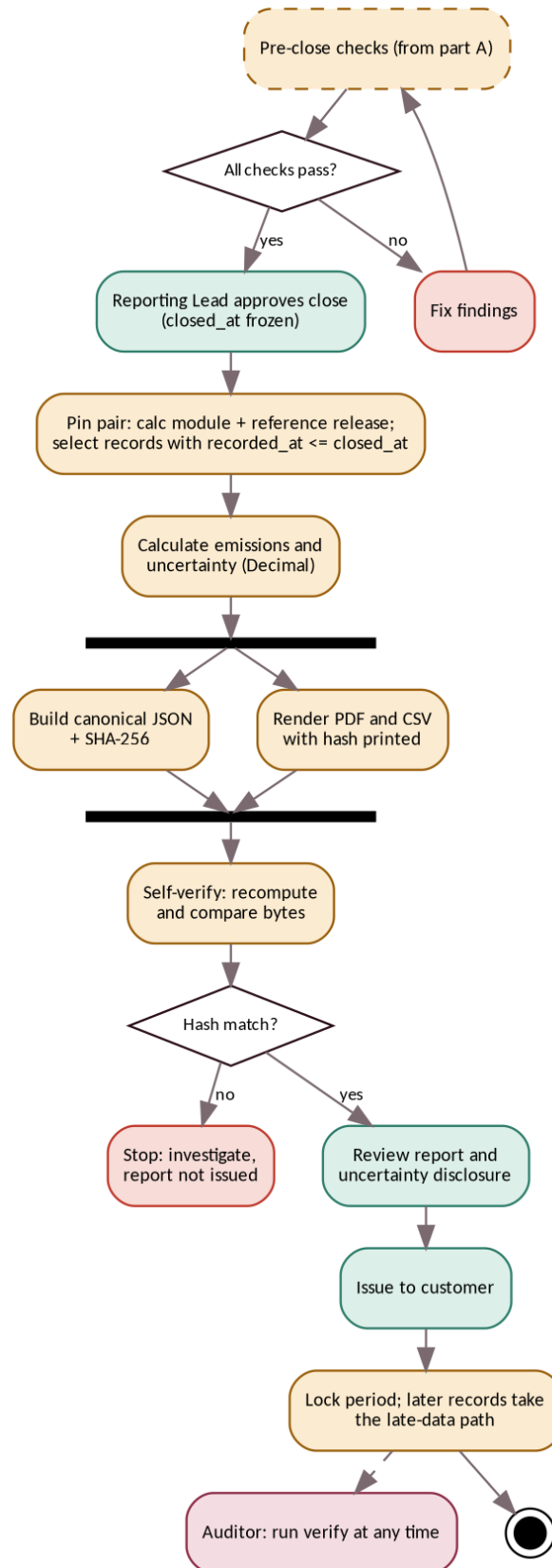


Figure 4b. Reporting cycle, part B: close, calculate in parallel with rendering, verify, issue, lock. Black bars are fork and join.

5. Behaviour design

5.1 Late record flow

The sequence of Figure 2 in the SRS is implemented by `close/late.py`. It runs in one database transaction: store record, tag periods, compute the factor from the release pinned by the attributed period's report, compute the adjustment against the amount most recently reported, queue the adjustment line, write the audit entry, notify. If any step fails the record is not stored and the user sees the error, so a half-processed late record cannot exist.

5.2 Calculation (pure function)

```
def calculate(records, release, policy):          # no clock, no database, no globals
    lines = []
    for r in sorted(records, key=lambda r: (r.plant, r.scope, r.activity_type, r.record_id)):
        if r.kind == "DIRECT_EMISSION":
            e = q6(r.quantity); factor = None
        else:
            f = release.lookup(r.activity_type, r.unit_after_conversion, r.activity_date) # raises if 0
        or >1
            e = q6(convert(r.quantity, r.unit, f.unit, release) * f.value)
            u = q6(e * policy.tier_rate[r.tier])
            lines.append(Line(r, f, e, u))
    return lines                                # totals: exact decimal sums; U(rss) = sqrt(sum(u*u)); U(lin) = sum(u)
```

Records that straddle a month boundary are apportioned earlier by the ingest service, so each line has a single activity date.

5.3 Canonical report and hash

`canonical.py` builds a plain dictionary (no run time, host or user), converts every decimal to a fixed 6-decimal string, sorts keys, dumps with `separators=(",", ":")` and `ensure_ascii=False`, encodes UTF-8 and adds one `\n`. The SHA-256 of those bytes is the report hash, printed on the PDF and stored in the report row.

5.4 Uncertainty disclosure

Per aggregate the report prints: **E ± U(rss) (x%)**, the worst-case bound U(lin), the share of emissions by tier, lists of gap-filled and declared items, and the fixed method statement. U is rounded to 2 significant figures and E to the same place (FR-UNC-05). Example from fixture FX-02: *62,000 ± 1,400 kgCO₂e (2.2%); worst-case bound 2,300; metered 80.6%, invoiced 16.1%, estimated 3.2%.*

5.5 Period state machine

States are those of Figure 1 in the SRS. Transitions are methods on `ReportingPeriod`; each checks its preconditions and writes an audit entry. There is no method that moves CLOSED back to OPEN, so reopening is impossible in code as well as by policy.

6. Design decisions and trade-offs

Decision	Benefit	Cost accepted
Frozen calc modules, never edited	Old reports reproduce even if a defect is later found.	A defect is fixed in a new module and old periods change only by restatement. Some code duplication between v1 and v2.
Copy-on-release factor rows	Native constraint, simple lookup, hash per release.	A few hundred duplicated rows per year.
Hash over canonical JSON	Byte stable.	The PDF is a rendering and may differ in bytes between runs; it prints the JSON hash.
Server-rendered pages	Nothing in the browser to version or secure.	Less dynamic UI, acceptable for data entry forms.
Single repository for code and reference data	One tag versions both (CL-CMP-001).	Reference data changes go through code review. Accepted: that is the control wanted.
RSS plus worst-case bound	Honest range.	Two numbers to explain; the method statement does this.

7

Traceability

Requirement group	Design element
FR-IN	<code>ingest/</code> , tables <code>activity_record</code> , <code>supplier_declaration</code> , immutability trigger
FR-CALC, FR-UNC	<code>calc/v1/</code> , <code>rounding.py</code> , <code>UncertaintyDisclosure</code>
FR-FAC	<code>reference_release</code> , <code>emission_factor</code> with exclusion constraint, manifest hash
FR-PC, FR-LATE	<code>close/</code> (state machine, gapfill, late, restate), tables <code>reporting_period</code> , <code>report</code> , <code>report_line</code>
FR-RPT, NFR-REP	<code>report/canonical.py</code> , <code>verify/cli.py</code> , <code>release_pair</code> , golden tests
FR-AUD, FR-AC	<code>audit_log</code> table, role checks in <code>web/</code>